

Robot d'Audit RPA

UiPath StudioX - Mode Assiste (Attended)

Departement	Business Financial Systems (BFS) - Disneyland Paris
Auteur	Zarath Mougamadou - Financial Planning & Reporting Apprentice
Tutrice	Project Leader Financial Systems
Demarrage	2026
Statut	En cours de developpement
Technologies	UiPath StudioX . Excel . Python . Chrome . DBeaver . JDBC

Compte-rendu - Robot Audit RPA

Table des matieres

1. Contexte et Objectifs
2. Description du projet
3. Realisation
 - 3.1. Etape 1 - Sanitization de l'environnement
 - 3.2. Etape 2 - Configuration et selection de la semaine
 - 3.3. Etape 3 - Navigation et Audit Visuel
 - 3.4. Etape 4 - Extraction ODB via script Python
 - 3.5. Etape 5 - Pipeline ETL local
 - 3.6. Etape 6 - Cloture et sauvegarde
4. Complexites techniques et solutions retenues
5. Bilan et perspectives

1. Contexte et Objectifs

Dans le cadre des activités du département Business Financial Systems (BFS) de Disneyland Paris, des audits périodiques des accès utilisateurs sont réalisés sur plusieurs systèmes d'information critiques : K2, Longview, ODB et SRH. Ces audits visent à garantir la conformité des droits d'accès et à produire des preuves documentaires pour les équipes de contrôle interne.

Initialement, ces audits étaient réalisés manuellement : un opérateur se connectait successivement à chaque portail, prenait des captures d'écran, exportait les listes d'utilisateurs, puis consolidait toutes les données dans un fichier Excel maître. Ce processus était long, répétitif et expose aux erreurs humaines.

L'objectif de ce projet est de développer un robot RPA en mode Assisté (Attended) via UiPath StudioX pour automatiser intégralement ce processus d'audit. Le robot prend en charge :

- La navigation autonome sur l'ensemble des portails cibles.
- La réalisation des captures d'écran de début et de fin de liste (preuves visuelles d'audit).
- L'injection automatique de ces captures dans le fichier Excel maître.
- L'extraction directe de la base ODB via script Python/JDBC (sans export CSV manuel).
- La transformation et la consolidation des données dans les onglets Excel correspondants.
- Le nettoyage de l'environnement en début et fin d'exécution.

Le mode Assisté a été retenu pour permettre à l'opérateur de superviser l'exécution et d'intervenir si nécessaire, tout en conservant le bénéfice d'une automatisation complète des tâches répétitives.

2. Description du projet

Le robot d'audit RPA est developpe avec UiPath StudioX et s'execute en mode Assiste via l'UiPath Assistant. Il cible quatre systemes successifs et produit un fichier Excel consolide comprenant les captures d'ecran d'audit et les listes d'utilisateurs filtrees.

Membres de l'equipe :

Role	Nom
Financial Planning & Reporting Apprentice (developpeur)	Zarath Mougamadou
Project Leader Financial Systems (tutrice)	-
Consultant SAP BW	-

Technologies utilisees :

Technologie	Usage
UiPath StudioX	Developpement et orchestration du robot RPA
Google Chrome	Navigateur cible (contournement des popups Windows 11/Edge)
Microsoft Excel	Fichier maitre de consolidation des donnees d'audit
Python 3 + jpype	Connexion JDBC a la base Informix ODB, extraction et export Excel
keyring	Gestion securisee des credentials (pas de mot de passe en clair)
pandas + openpyxl	Construction et export du DataFrame en fichier .xlsx intermediaire
Script Batch / os.system	Sanitization - suppression des processus fantomes
DBeaver + JDBC/BSON	Audit croise des bases de donnees (usage developpeur)
IA Generative	Support au developpement et a la documentation

Systemes audites :

- K2 (portail de gestion des workflows)
- Longview (outil de planification financiere)
- ODB (base de donnees operationnelle Informix - acces direct via JDBC)
- SRH (systeme de gestion des ressources humaines)

3. Realisation

Le code est entierement centralise dans le fichier Main.xaml. L'architecture repose sur un design pattern dit 'Single State Wrapper' qui garantit une gestion robuste de l'interface Excel et elimine les erreurs de pont COM (RCW). Le pipeline complet se deroule en six etapes sequentielles. Deux scripts Python auxiliaires (Kill_Processes.py et Extract_ODB_Users.py) completent le workflow UiPath.

3.1. Etape 1 - Sanitization de l'environnement

Avant toute execution, le script Kill_Processes.py est lance. Il termine de force les processus des navigateurs (msedge.exe, chrome.exe, firefox.exe) potentiellement ouverts en arriere-plan depuis une execution precedente. Le flag /F force la fermeture, /T inclut les processus enfants.

Kill_Processes.py
<pre>import os</pre>
<pre># /F : force la fermeture /T : processus enfants /IM : nom du processus</pre>
<pre>os.system("taskkill /F /T /IM msedge.exe")</pre>
<pre>os.system("taskkill /F /T /IM chrome.exe")</pre>
<pre>os.system("taskkill /F /T /IM firefox.exe")</pre>

Script de nettoyage lance avant chaque execution du robot.

3.2. Etape 2 - Configuration et selection de la semaine

Une boite de dialogue (Input Dialog) est presentee a l'utilisateur au demarrage. Elle lui permet de choisir la semaine d'execution : S1 ou S2. Ce choix conditionne l'aiguillage du flux de travail via un bloc If, permettant ainsi d'adapter le comportement du robot selon le calendrier d'audit en vigueur.

3.3. Etape 3 - Navigation et Audit Visuel

Pour chaque systeme audite, le robot realise la sequence suivante :

- Ouverture du portail dans Google Chrome.
- Prise d'une capture d'ecran en haut de liste (preuve de debut d'audit).
- Defilement automatique vers le bas (raccourci End ou boucle Page Down + Check App State).
- Prise d'une capture d'ecran en bas de liste (preuve de fin d'audit).
- Injection des deux captures dans Excel via un fichier image temporaire (contournement du presse-papiers).

3.4. Etape 4 - Extraction ODB via script Python

Pour le systeme ODB (base Informix), l'extraction ne passe pas par un export CSV depuis un portail web, mais par une connexion directe a la base de donnees via un driver JDBC, orchestre par le script Extract_ODB_Users.py. UiPath appelle ce script en tant que processus externe et attend son execution complete avant de poursuivre.

Detection intelligente du repertoire de travail :

Le script gere deux contextes d'execution differents avec un bloc try/except :

Extract_ODB_Users.py - Detection du contexte d'execution

```
try:
    # Mode CMD : le script est dans le sous-dossier 'Scripts/'
    dossier_actuel = os.path.dirname(os.path.abspath(__file__))
    if os.path.basename(dossier_actuel).lower() == 'scripts':
        dossier_racine = os.path.dirname(dossier_actuel)
    else:
        dossier_racine = dossier_actuel
except NameError:
    # Mode UiPath : le robot execute toujours depuis la racine du projet
    dossier_racine = os.getcwd()

dossier_drivers = os.path.join(dossier_racine, 'Drivers')
dossier_temp = os.path.join(dossier_racine, 'Temp')

__file__ n'est pas defini quand Python est appele par UiPath, d'ou le fallback sur os.getcwd().
```

Connexion JDBC a la base Informix ODB :

Le mot de passe est recupere depuis le gestionnaire de credentials Windows via keyring (aucun secret en clair dans le code). La JVM est demarree une seule fois via jpye, avec le driver JDBC Informix charge en classpath.

Extract_ODB_Users.py - Connexion JDBC Informix

```
# Recuperation securisee du mot de passe (Windows Credential Manager)
utilisateur_courant = getpass.getuser().lower()
mot_de_passe = keyring.get_password("Informix", utilisateur_courant)

# Demarrage de la JVM (une seule fois par session)
if not jpye.isJVMStarted():
    jpye.startJVM(chemin_jvm, '-ea',
        classpath=[chemin_jar_informix, chemin_jar_bson])

# Chargement du driver et connexion
jpye.JClass("com.informix.jdbc.IfxDriver")
DriverManager = jpye.JClass('java.sql.DriverManager')
conn = DriverManager.getConnection(url, utilisateur_courant, mot_de_passe)
```

Requete SQL et extraction des donnees :

Extract_ODB_Users.py - Requete SQL et lecture ResultSet

```
requete_sql = (
    "select t300_user_name, t300_cast_id, t300_last_login,"
    " t300_profile_code, t300_netuser"
```

```

" from t300_user order by t300_user_name;"
)

stmt = conn.createStatement()
rs = stmt.executeQuery(requete_sql)

# Extraction des metadonnees de colonnes
meta = rs.getMetaData()
nb_col = meta.getColumnCount()
colonnes = [str(meta.getColumnName(i)) for i in range(1, nb_col + 1)]

# Lecture ligne par ligne du ResultSet
lignes = []
while rs.next():
    ligne = [str(rs.getString(i)) if rs.getString(i) is not None else ''
              for i in range(1, nb_col + 1)]
    lignes.append(ligne)

rs.close() ; stmt.close() ; conn.close()

```

Export vers fichier Excel intermediaire :

Le DataFrame pandas est ecrit dans un fichier odb_users.xlsx dans le repertoire Temp. UiPath lit ensuite ce fichier via une activite Read Range pour l'injecter dans l'onglet Excel maitre. En cas d'echec, le traceback complet est ecrit dans un fichier erreur_python.txt pour faciliter le diagnostic.

Extract_ODB_Users.py - Export Excel et gestion d'erreur

```

# Construction du DataFrame et export Excel
df = pd.DataFrame(lignes, columns=colonnes)
odb_users_filepath = os.path.join(dossier_temp, 'odb_users.xlsx')
df.to_excel(odb_users_filepath, index=False, engine='openpyxl')

# Gestion d'erreur : ecriture du traceback pour diagnostic UiPath
except Exception as e:
    chemin_erreur = os.path.join(dossier_temp, 'erreur_python.txt')
    with open(chemin_erreur, 'w') as f:
        f.write(traceback.format_exc())
    sys.exit(1) # Code retour non-zero => UiPath detecte l'echec

```

sys.exit(1) permet a UiPath de detecter l'echec du script et d'interrompre le workflow proprement.

3.5. Etape 5 - Pipeline ETL local

Chaque fichier CSV (ou xlsx pour ODB) telecharge est traite par le pipeline ETL local suivant :

Operation	Activite UiPath	Detail
Extract	Read CSV / Read Range	Lecture du fichier source. Delimiteur configure strictement.
Transform	Filter DataTable	Filtrage par index de colonnes (robustesse aux renommages).

Load	Write Range	Injection dans l'onglet Excel dedie au systeme audite.
------	-------------	--

3.6. Etape 6 - Cloture et sauvegarde

En fin d'execution, le robot procede dans l'ordre suivant : fermeture des onglets Chrome, sauvegarde native du fichier Excel maitre, puis vidage du repertoire Temp local (fichiers CSV, xlsx et images intermediaires).

4. Complexites techniques et solutions retenues

Le developpement du robot a necessite l'identification et la resolution de plusieurs problemes techniques non triviaux. Les regles suivantes constituent la base de connaissance de l'outil et doivent etre respectees lors de toute evolution.

Choix du navigateur : Chrome vs Edge

Sur Windows 11, le navigateur Edge declenche des popups de securite 'Flyout' qui bloquent les telechargements silencieux. Google Chrome, avec l'extension UiPath activee, ne presente pas ce comportement et a ete retenu comme navigateur cible unique.

Gestion du defilement (scrolling)

Le defilement via la molette de la souris est proscriit. Deux strategies : (1) Pour les listes statiques : clic de focus + raccourci End. (2) Pour le Lazy Loading : boucle Repeat avec Page Down securisee par Check App State.

Robustesse des filtres de donnees

Les filtres sur les DataTables utilisent les index numeriques de colonnes (0, 1, 2) et non les en-tetes textuels, pour resister aux renommages dans les fichiers sources.

Injection des captures d'ecran dans Excel

Le robot ecrit la capture dans un fichier image temporaire (repertoire Temp), puis la lit et l'insere dans Excel via son chemin de fichier, contournant ainsi le presse-papiers Windows.

Pattern Single State Wrapper (COM Excel)

L'interface Excel est ouverte une seule fois via une carte globale Use Excel File. Tous les blocs de traitement y sont imbriquees. La carte se ferme en toute fin de workflow.

Gestion duale du contexte Python (CMD vs UiPath)

Le script Extract_ODB_Users.py detecte son contexte d'execution via un try/except sur `__file__`. En mode CMD, il remonte depuis le sous-dossier Scripts/. En mode UiPath, il utilise `os.getcwd()` puisque `__file__` n'est pas defini. Cela garantit la portabilite du script sans modification.

Securite des credentials

Aucun mot de passe n'est ecrit en clair dans le code. Les credentials Informix sont stockes dans le gestionnaire Windows et recuperes via `keyring.get_password()` au moment de l'execution.

5. Bilan et perspectives

Le robot d'audit RPA permet de reduire significativement le temps et les efforts necessaires a la realisation des audits d'accès periodiques. L'automatisation des taches de navigation, de capture et de consolidation garantit une coherence et une fiabilite superieures a l'approche manuelle anterieure.

Acquis techniques :

- Maitrise du developpement RPA avec UiPath StudioX en environnement de production.
- Conception d'un pattern d'architecture robuste (Single State Wrapper) adapte aux contraintes COM Excel.
- Developpement d'un connecteur Python/JDBC pour acces direct a une base Informix.
- Mise en place d'une gestion securisee des credentials via keyring.
- Resolution de problemes lies aux interfaces web dynamiques (Lazy Loading, popups systeme).

Axes d'amelioration envisages :

- Extension du perimetre a d'autres systemes audites.
- Migration du script de sanitization vers une activite UiPath native.
- Ajout d'un rapport d'execution automatique (log des actions et erreurs) genere en fin de run.
- Exploration d'une execution en mode Non Assiste (Unattended) via UiPath Orchestrator.